

MarIA — Documentação Técnica Completa

Assistente de IA Católico com integração WhatsApp, gestão de assinaturas e painel administrativo.

Versão: 1.0 | **Data:** Junho 2026 | **Stack:** NestJS · React/Vite · Next.js · Supabase

Índice

- Visão Geral do Projeto
- Análise de Requisitos
- Arquitetura do Sistema
- Estrutura de Diretórios
- Schema do Banco de Dados
- Mapeamento de Endpoints
- Webhooks
- Integrações Externas
- Fluxos de Dados
- Autenticação e Autorização
- Sistema de Cache
- Sistema de Cotas e Assinaturas
- Variáveis de Ambiente
- Frontend — Painel Administrativo
- Landing Page
- Segurança
- Observabilidade e Logs
- Infraestrutura e Deploy

1. Visão Geral do Projeto

MarIA é uma assistente de IA com temática católica, acessível via WhatsApp. O sistema oferece:

- Respostas teológicas fundamentadas no Magistério da Igreja
- Conteúdo litúrgico diário (liturgia, santo do dia, mistérios do rosário)
- Sistema de assinatura pago com tiers (básico e premium)
- Portal do cliente para gerenciamento de plano
- Painel administrativo completo para operação do negócio

Stack Tecnológica

Camada	Tecnologia
Backend API	NestJS (TypeScript)
Banco de Dados	Supabase (PostgreSQL + pgvector)
Admin Frontend	React 19 + Vite + Tailwind CSS v4
Landing Page	Next.js 14 (App Router)
WhatsApp	UAZAPI
Pagamentos	Asaas
IA Principal	OpenRouter (GPT-4o-mini / Gemini Flash)
IA Teológica	Magisterium AI
E-mail	Nodemailer (SMTP)
Process Manager	PM2

2. Análise de Requisitos

2.1 Requisitos Funcionais

RF01 — INTERAÇÃO VIA WHATSAPP

- O sistema deve receber mensagens de texto via webhook UAZAPI
- Deve ignorar mensagens de grupos e mensagens enviadas pelo próprio bot
- Deve recusar mensagens de áudio com uma mensagem amigável
- Deve suportar botões interativos com fallback para lista numerada

RF02 — TRIAGEM DE NOVOS USUÁRIOS

- Novos usuários devem passar por um fluxo de triagem:
 1. Apresentação da MarIA
 2. Coleta do nome do usuário
 3. Apresentação dos planos de assinatura

RF03 — PROCESSAMENTO DE MENSAGENS COM IA

- O sistema deve classificar a intenção da mensagem (intent router)
- Mensagens sobre liturgia, santo do dia e rosário devem usar cache diário (zero custo LLM)
- Mensagens teológicas complexas devem consultar a base Magisterium AI
- Demais mensagens devem ser processadas pelo LLM principal

RF04 — CACHE E CONTEÚDO DIÁRIO

- Um job CRON deve gerar diariamente (00:01 horário de Brasília):
- Liturgia do dia
- Santo do dia
- Mistérios do rosário
- Conteúdo deve ser editável pelo admin via painel

RF05 — SISTEMA DE ASSINATURA (WHATSAPP)

- Usuário deve poder assinar direto pelo WhatsApp via fluxo conversacional
- Fluxo: selecionar plano → selecionar ciclo → receber link de pagamento Asaas
- Após pagamento confirmado: gerar código de ativação (MARIA-XXXXXX)
- Enviar código por e-mail e/ou WhatsApp
- Usuário digita o código no WhatsApp para ativar

RF06 — SISTEMA DE ASSINATURA (WEB)

- Landing page deve oferecer formulário de checkout
- Integração direta com Asaas para criação de link de pagamento
- Polling do status do pagamento pela landing page
- Exibição do código de ativação após confirmação
- Redirecionamento para WhatsApp com código pré-preenchido

RF07 — PORTAL DO CLIENTE

- Cliente deve poder fazer login via código verificador enviado ao WhatsApp
- Visualizar status da assinatura, tier, validade
- Cancelar assinatura
- Trocar de plano

RF08 — PAINEL ADMINISTRATIVO

- Listar e gerenciar usuários WhatsApp (status, tier, histórico)
- Ver e editar prompts do sistema
- Gerenciar fluxos automáticos (botões, passos)
- Dashboard financeiro (receita, custo, margem)
- Ver e editar cache diário
- Configurações do sistema (modelos, modo manutenção)
- Logs de uso e webhooks
- Gestão de admins (criar, editar, excluir)

RF09 — MEMÓRIA CONTEXTUAL

- A cada 10 mensagens, condensar contexto do usuário em resumo
- Extrair interesses e preferências do usuário
- Injetar contexto nas próximas interações para personalização

RF10 — CACHE SEMÂNTICO

- Armazenar respostas teológicas com embeddings vetoriais
- Reutilizar respostas semelhantes (similaridade ≥ 0.92) sem chamar LLM

2.2 Requisitos Não-Funcionais

RNF01 — PERFORMANCE

- Webhook deve retornar resposta imediatamente (processamento assíncrono)
- Cache diário elimina latência e custo de LLM para conteúdo previsível

RNF02 — DISPONIBILIDADE

- Modo manutenção configurável pelo admin
- PM2 para processo sempre ativo em produção

RNF03 — SEGURANÇA

- Rate limiting em todos os endpoints sensíveis
- Validação de token nos webhooks Asaas
- RLS (Row Level Security) no Supabase
- Auditoria de todas as ações administrativas

RNF04 — ESCALABILIDADE

- Cache semântico reduz chamadas ao LLM
- Cache diário pré-gerado elimina pico de chamadas

RNF05 — OBSERVABILIDADE

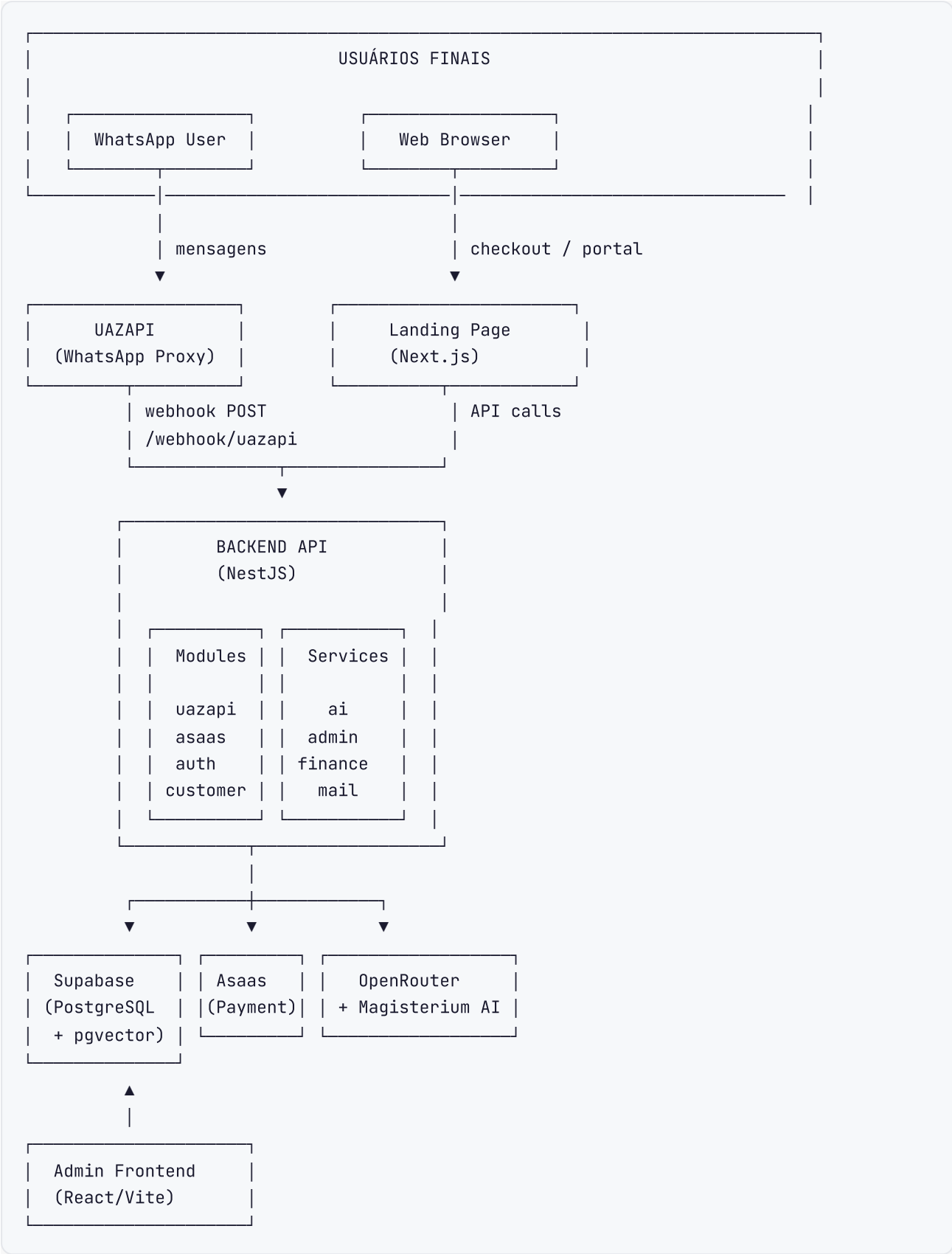
- Log de uso (tokens, modelo, custo) por mensagem
- Log de webhooks recebidos
- Log de auditoria de ações administrativas

2.3 Planos e Preços

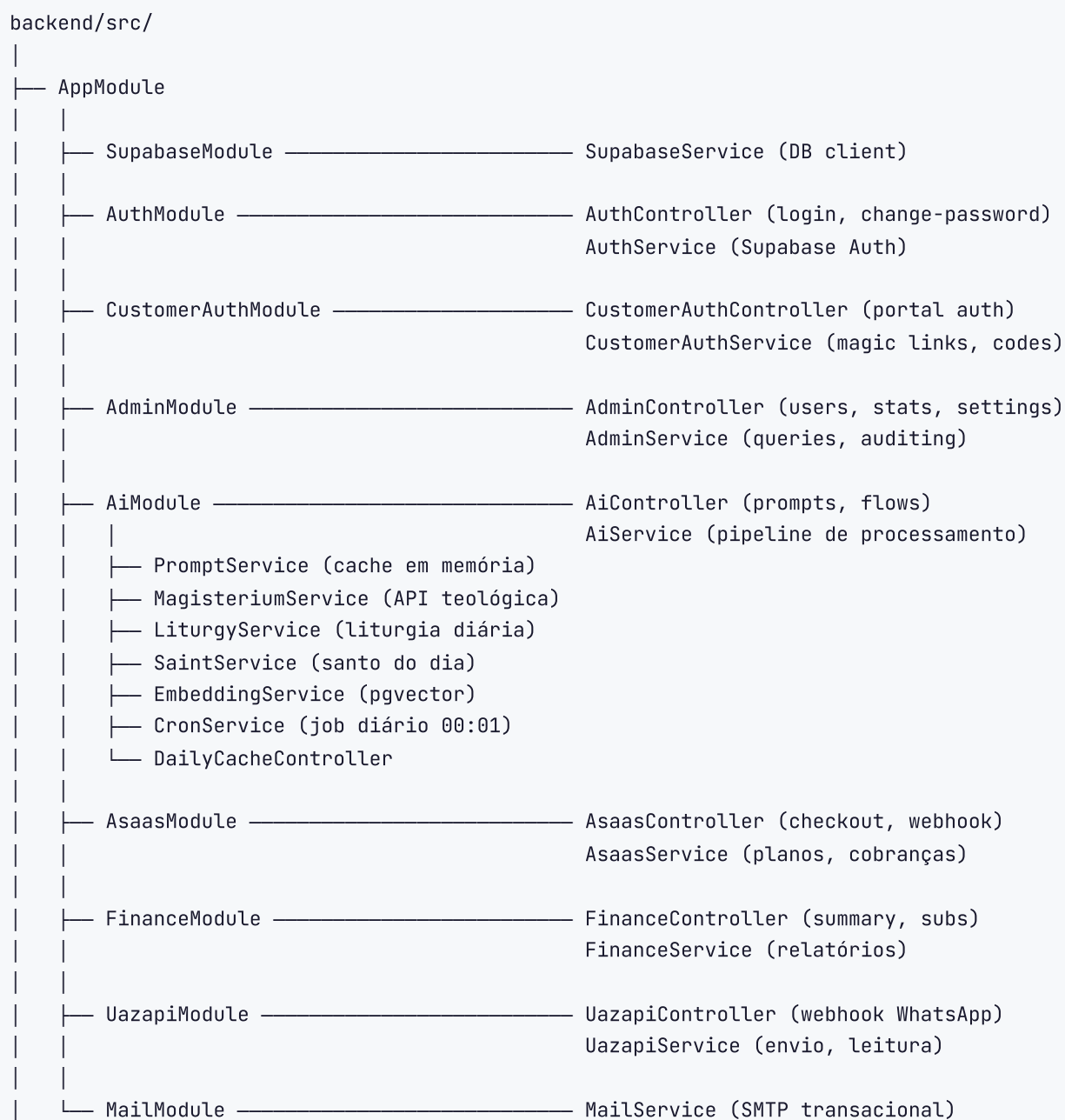
Plano	Ciclo	Preço	Mensagens LLM/mês
Gratuito	—	R\$ 0	0 (apenas cache)
Básico	Mensal	R\$ 14,90	300
Básico	Anual	R\$ 154,80	300
Premium	Mensal	R\$ 29,90	600
Premium	Anual	R\$ 322,80	600
Admin	—	—	Ilimitado

3. Arquitetura do Sistema

3.1 Diagrama de Alto Nível

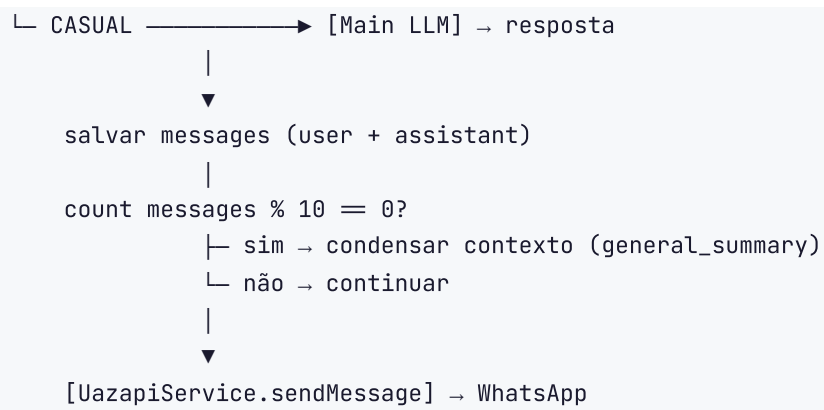


3.2 Diagrama de Módulos Backend



3.3 Diagrama de Pipeline de Mensagem





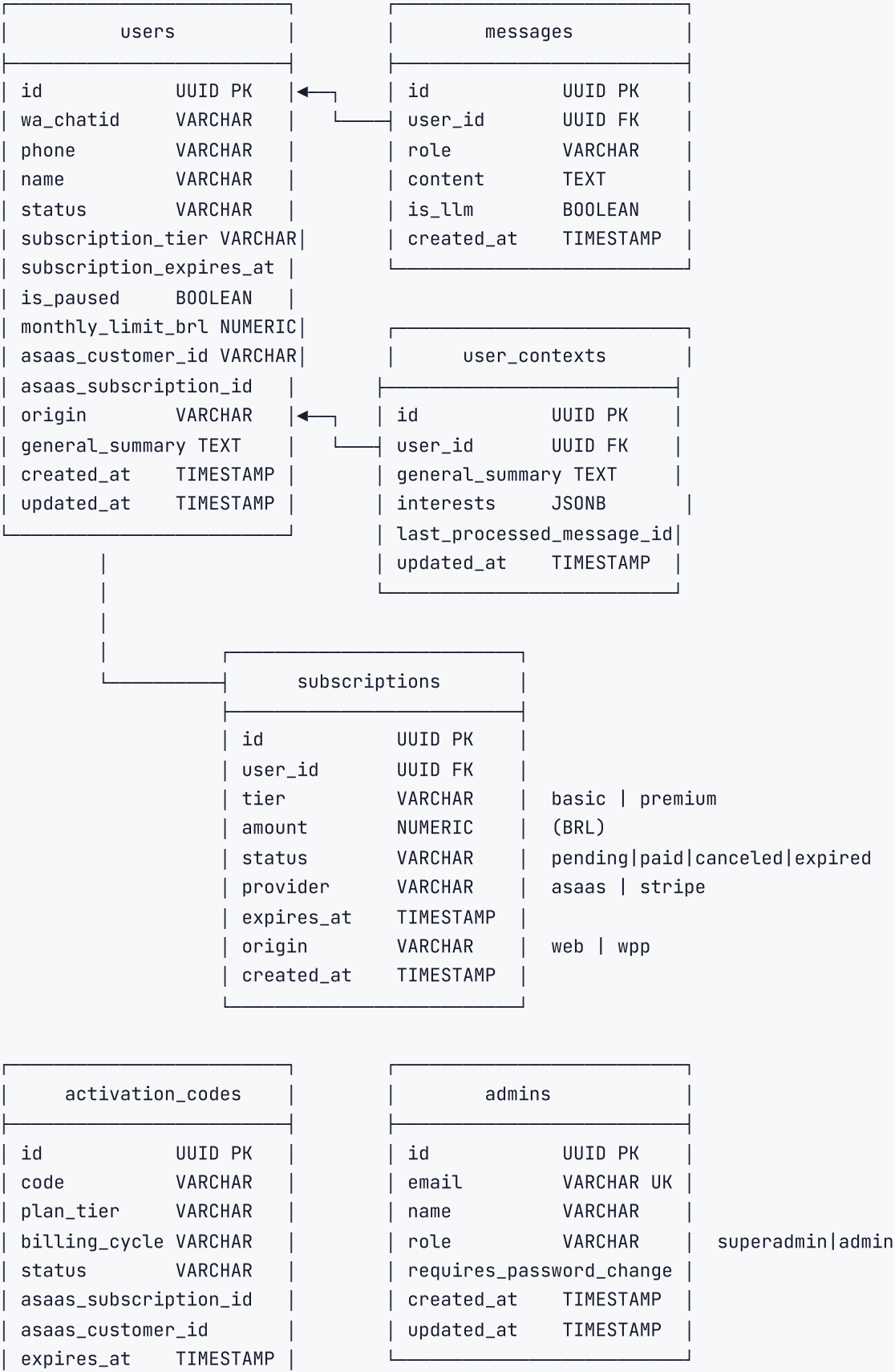
4. Estrutura de Diretórios

```
MarIA/
├── backend/                                # API NestJS
│   ├── src/
│   │   ├── admin/                        # Módulo administrativo
│   │   ├── ai/                          # Motor de IA e cache
│   │   ├── asaas/                       # Integração pagamentos
│   │   ├── auth/                        # Autenticação admin
│   │   ├── customer-auth/              # Portal do cliente
│   │   ├── finance/                    # Módulo financeiro
│   │   ├── mail/                       # Serviço de e-mail
│   │   ├── stripe/                     # Placeholder Stripe (legado)
│   │   ├── supabase/                   # Cliente do banco
│   │   ├── uazapi/                     # WhatsApp integration
│   │   ├── app.module.ts               # Módulo raiz
│   │   └── main.ts                     # Bootstrap
│   ├── .env.example
│   ├── nest-cli.json
│   ├── package.json
│   └── tsconfig.json
├── frontend/                              # Painei Admin (React/Vite)
│   ├── src/
│   │   ├── components/
│   │   │   ├── ui/                     # Shadcn UI primitivos
│   │   │   ├── layout/                 # Wrapper + Sidebar
│   │   │   └── dashboard/              # KPI cards
│   │   ├── pages/
│   │   │   ├── login.tsx
│   │   │   ├── dashboard.tsx
│   │   │   ├── wa-users.tsx
│   │   │   ├── ai-settings.tsx
│   │   │   ├── flows.tsx
│   │   │   ├── daily-content.tsx
│   │   │   ├── finance.tsx
│   │   │   ├── settings.tsx
│   │   │   └── logs.tsx
│   │   ├── lib/
│   │   │   ├── api.ts                  # Fetch wrapper
│   │   │   └── utils.ts
│   │   ├── App.tsx                     # Roteamento
│   │   └── main.tsx
│   ├── .env.development
│   ├── .env.production
│   ├── vite.config.ts
│   └── package.json
├── landing/                              # Site público (Next.js)
│   └── src/app/
│       ├── page.tsx                    # Home + checkout
│       └── portal/                      # Portal do cliente
```

```
|      └─ ...
|
|─ docs/                                # Documentação
|   └─ DOCUMENTATION.md                # Este arquivo
|
|─ package.json                        # Scripts orquestradores
|─ ecosystem.config.js                 # Configuração PM2
|─ deploy.sh                           # Script de deploy
|─ dev.ps1                             # Dev script Windows
└─ CHANGELOG.md
```

5. Schema do Banco de Dados

5.1 Diagrama Entidade-Relacionamento



used_at	TIMESTAMP
wa_chatid	VARCHAR
created_at	TIMESTAMP

ai_prompts	
id	UUID PK
key	VARCHAR UK
content	TEXT
description	TEXT
is_active	BOOLEAN
updated_at	TIMESTAMP

system_settings	
id	UUID PK
key	VARCHAR UK
value	TEXT
updated_at	TIMESTAMP

automatic_flows	
id	UUID PK
key	VARCHAR UK
name	VARCHAR
steps	JSONB
updated_at	TIMESTAMP

magic_links	
id	UUID PK
user_id	UUID FK
token	VARCHAR UK
expires_at	TIMESTAMP
used	BOOLEAN
created_at	TIMESTAMP

magisterium_cache	
id	UUID PK
question	TEXT
answer	TEXT
intent	VARCHAR

FK admin_id



activity_logs	
id	UUID PK
admin_id	UUID
admin_email	VARCHAR
admin_name	VARCHAR
action	VARCHAR
details	JSONB
created_at	TIMESTAMP

daily_cache	
id	UUID PK
type	VARCHAR
cache_date	DATE
content	TEXT
updated_at	TIMESTAMP

usage_logs	
id	UUID PK
user_id	UUID FK
model	VARCHAR
prompt_tokens	INT
completion_tokens	INT
total_tokens	INT
cost	NUMERIC
created_at	TIMESTAMP

webhook_logs	
id	UUID PK
event_type	VARCHAR
payload	JSONB
created_at	TIMESTAMP

liturgy|saint|rosary

(USD)

embedding	vector	pgvector – 1536 dims
created_at	TIMESTAMP	

5.2 Definição de Tipos

ENUM: **SUBSCRIPTION_TIER**

free | basic | premium | unlimited | admin

ENUM: **USER_STATUS**

```

trriage_intro           # novo usuário, apresentação inicial
trriage_name            # aguardando nome do usuário
trriage_presentation_subscription # apresentando opções de plano
active                  # usuário ativo, processamento normal
disabled                # usuário desabilitado
flow:<flow_key>:<step_id> # dentro de um fluxo automático (ex: flow:subscription_flow)

```

ENUM: **SUBSCRIPTION_STATUS**

pending | paid | canceled | expired

ENUM: **ADMIN_ROLE**

superadmin | admin

STRUCTURE: **AUTOMATIC_FLOWS.STEPS** (JSONB)

```

{
  "select_plan": {
    "text": "Qual plano você gostaria?",
    "buttons": [
      { "id": "basic", "text": "Básico", "keywords": ["básico", "basico"] },
      { "id": "premium", "text": "Premium", "keywords": ["premium"] }
    ]
  },
  "select_cycle": {
    "text": "Qual ciclo de pagamento?",
    "buttons": [
      { "id": "monthly", "text": "Mensal" },
      { "id": "annual", "text": "Anual" }
    ]
  }
}

```

5.3 Índices

```
CREATE INDEX idx_users_asaas_customer_id ON users(asaas_customer_id);
CREATE INDEX idx_users_asaas_subscription_id ON users(asaas_subscription_id);
CREATE INDEX idx_magic_links_token ON magic_links(token);
-- pgvector index para busca semântica
CREATE INDEX idx_magisterium_embedding ON magisterium_cache USING ivfflat (embedding
```

6. Mapeamento de Endpoints

6.1 Autenticação Admin

Método	Endpoint	Body	Descrição
POST	/auth/login	{ email, password }	Login admin — retorna JWT Supabase
POST	/auth/change-password	{ email, password, newPassword }	Trocar senha (obrigatório no primeiro login)

Rate limit: 3 req/min

6.2 Painel Administrativo — Usuários

Header obrigatório: x-admin-id: <uuid>

Método	Endpoint	Query/Body	Descrição
GET	/admin/users	—	Listar admins cadastrados
GET	/admin/wa-users	—	Listar usuários WhatsApp com métricas
GET	/admin/wa-users/:id/messages	—	Histórico de conversa do usuário
POST	/admin/wa-users/:id/clear-data	—	Limpar dados do usuário (histórico, contexto)
POST	/admin/wa-users/:id/subscription	{ tier, expiresAt }	Atualizar tier/validade manualmente
POST	/admin/wa-users/:id/settings	{ isPaused, monthlyLimitBrl }	Pausar usuário ou definir limite de gasto

6.3 Painel Administrativo — Estatísticas

Método	Endpoint	Query	Descrição
GET	/admin/stats	—	Estatísticas gerais do dashboard
GET	/admin/stats/daily	startDate?, endDate?	Estatísticas diárias por período
GET	/admin/activities	targetId?	Timeline de auditoria de ações

6.4 Painel Administrativo — Configurações

Header: x-admin-id: <uuid>

Método	Endpoint	Body	Descrição
GET	/admin/settings	—	Listar todas as configurações do sistema
GET	/admin/settings/public/:key	—	Buscar configuração pública por chave
PATCH	/admin/settings/:key	{ value }	Atualizar configuração por chave
POST	/admin/settings/sync-exchange	—	Sincronizar taxa de câmbio BRL/USD
POST	/admin/settings/clear-cache	—	Limpar cache semântico (magisterium_cache)
POST	/admin/settings/toggle-maintenance	—	Ativar/desativar modo manutenção
GET	/admin/ai-models	—	Listar modelos de IA disponíveis

Chaves de configuração relevantes:

Chave	Tipo	Descrição
main_model	string	Modelo LLM principal (ex: openai/gpt-4o-mini)
bridge_model	string	Modelo de intent routing (ex: google/gemini-2.0-flash-lite)
maintenance_mode	boolean (string)	Ativar modo de manutenção
brl_rate	number (string)	Taxa de câmbio USD→BRL

6.5 Painel Administrativo — Gestão de Admins

Header: x-admin-id: <uuid> (superadmin apenas para criação/exclusão)

Método	Endpoint	Body	Descrição
POST	/admin/admins	{ email, name, role }	Criar novo admin
PATCH	/admin/admins/:id	{ name?, role?, password? }	Atualizar admin
DELETE	/admin/admins/:id	—	Excluir admin

6.6 Painel Administrativo — Logs

Método	Endpoint	Query	Descrição
GET	/admin/logs/usage	page?, limit?, startDate?, endDate?	Logs de tokens por modelo/usuário
GET	/admin/logs/webhooks	page?, limit?, startDate?, endDate?	Logs de webhooks recebidos

6.7 Configuração de IA

Método	Endpoint	Body	Descrição
GET	/ai/prompts	—	Listar todos os prompts ativos
PUT	/ai/prompts/:key	{ content, description?, is_active? }	Atualizar prompt por chave
POST	/ai/prompts/generate	{ key, description, currentContent? }	Gerar prompt via GPT-4o
GET	/ai/automatic-flows	—	Listar todos os fluxos automáticos
PUT	/ai/automatic-flows/:key	{ steps, name? }	Atualizar fluxo

Chaves de prompt relevantes:

Chave	Descrição
core_persona	Personalidade principal da MarIA
intent_router	Instrução de classificação de intenção
extractor_name	Extração de nome durante triagem
context_condenser	Condensação de contexto do usuário

6.8 Cache Diário

Método	Endpoint	Body/Query	Descrição
GET	/ai/daily-cache	date? (YYYY-MM-DD)	Buscar cache para uma data
PUT	/ai/daily-cache/:id	{ content }	Editar item do cache
POST	/ai/daily-cache/generate	{ date, force?, type? }	Gerar cache manualmente

Tipos de cache: `liturgy` · `saint` · `rosary`

6.9 Financeiro

Método	Endpoint	Query/Body	Descrição
GET	/admin/finance/summary	startDate?, endDate?	Resumo financeiro (receita, custo, margem)
GET	/admin/finance/subscriptions	limit?, offset?, startDate?, endDate?	Listar assinaturas paginadas
POST	/admin/finance/record-manual	{ userId, tier, amount }	Registrar pagamento manual
POST	/admin/finance/subscriptions/:id/cancel	—	Cancelar assinatura
DELETE	/admin/finance/subscriptions/:id	—	Excluir registro de assinatura
POST	/admin/finance/sync-asaas	—	Sincronizar assinaturas com Asaas
POST	/admin/finance/subscriptions/:id/update	{ tier, cycle }	Alterar plano/ciclo

Header para operações financeiras: `x-admin-email: <email>`

6.10 Pagamentos (Asaas)

Método	Endpoint	Body	Descrição
POST	/payment/asaas/checkout	{ planId, cycle, phone }	Criar link de checkout (fluxo WhatsApp)
POST	/payment/asaas/checkout-web	{ planId, cycle }	Criar sessão de checkout (fluxo Web)

Método	Endpoint	Body	Descrição
GET	/payment/asaas/status/:sessionId	—	Verificar status do pagamento web (polling)
POST	/payment/asaas/webhook	payload Asaas	Receber notificações de pagamento

IDs de plano:

planId	Tier	Ciclo	Preço
basic_monthly	basic	monthly	R\$ 14,90
basic_annual	basic	annual	R\$ 154,80
premium_monthly	premium	monthly	R\$ 29,90
premium_annual	premium	annual	R\$ 322,80

6.11 Portal do Cliente

Rate limit: 5 req/min (verificação de código), 3 req/min (envio de código)

Método	Endpoint	Body	Auth	Descrição
POST	/customer/auth/send-code	{ phone }	Pública	Enviar código 6 dígitos via WhatsApp
POST	/customer/auth/verify-code	{ phone, code }	Pública	Verificar código e retornar token
POST	/customer/auth/magic-link	{ phone }	Pública	(Legado) Enviar magic link
POST	/customer/auth/verify	{ token }	Pública	(Legado) Verificar magic link
GET	/customer/subscription/status	—	Bearer	Dados completos da assinatura
POST	/customer/subscription/cancel	—	Bearer	Cancelar assinatura
POST	/customer/subscription/change-plan	{ planId, cycle }	Bearer	Trocar de plano

6.12 WhatsApp Webhook

Método	Endpoint	Descrição
POST	/webhook/uazapi	Receber mensagens do UAZAPI

7. Webhooks

7.1 UAZAPI → MarIA (POST /webhook/uazapi)

Origem: UAZAPI (proxy WhatsApp)

Autenticação: Nenhuma (confiança por isolamento de rede/IP)

Payload esperado:

```
{
  "EventType": "messages",
  "message": {
    "key": {
      "remoteJid": "5562981234567@s.whatsapp.net",
      "fromMe": false
    },
    "messageTimestamp": 1749600000,
    "message": {
      "conversation": "Texto da mensagem",
      "extendedTextMessage": { "text": "Texto com formatação" },
      "buttonsResponseMessage": { "selectedButtonId": "basic" },
      "listResponseMessage": {
        "singleSelectReply": { "selectedRowId": "premium" }
      },
      "audioMessage": {}
    },
    "senderName": "Nome do Usuário"
  },
  "chat": {
    "phone": "5562981234567",
    "wa_name": "Nome WhatsApp",
    "wa_contactName": "Nome do Contato"
  }
}
```

Lógica de processamento:

1. Identificar tipo de mensagem (texto / botão / lista / áudio)
2. Extrair chatId (remoteJid) e phone normalizado
3. fromMe == true → **IGNORAR**
4. chatId termina em @g.us → **IGNORAR** (grupo)
5. audioMessage presente → Responder recusa amigável

6. Texto corresponde a `MARIA-[A-Z0-9]{6}` → Validar código de ativação

7. Demais casos → Encaminhar para `AiService.processMessage()`

Resposta HTTP: `200 OK` imediato (processamento assíncrono)

7.2 Asaas → MarIA (`POST /payment/asaas/webhook`)

Origem: Asaas (plataforma de pagamentos)

Autenticação: Header `asaas-access-token` validado contra `ASAAS_AUTH_TOKEN`

Eventos tratados:

Evento	Ação
<code>PAYMENT_RECEIVED</code> / <code>PAYMENT_CONFIRMED</code>	Criar <code>activation_code</code> (MARIA-XXXXXX), enviar por e-mail, atualizar assinatura
<code>PAYMENT_OVERDUE</code>	Marcar assinatura como <code>expired</code>
<code>SUBSCRIPTION_CANCELED</code>	Cancelar assinatura, reverter tier para <code>free</code>
<code>SUBSCRIPTION_EXPIRED</code>	Expirar assinatura

Payload de exemplo (PAYMENT_RECEIVED):

```
{
  "event": "PAYMENT_RECEIVED",
  "payment": {
    "id": "pay_123456789",
    "customer": "cus_ABC123",
    "subscription": "sub_XYZ789",
    "value": 14.90,
    "status": "RECEIVED",
    "externalReference": "basic_monthly|5562981234567"
  }
}
```

Formato do campo `externalReference` : `<planId>|<phone>`

Fluxo após pagamento confirmado:

```
webhook recebido
|
▼
validar token (asaas-access-token)
|
▼
buscar usuário por asaas_customer_id
|
▼
gerar código: MARIA-[RANDOM 6 chars A-Z0-9]
|
▼
salvar activation_codes (status: pending, expires: 48h)
|
├─> enviar e-mail com código (MailService)
└─> opcionalmente enviar via WhatsApp (UazapiService)
```

8. Integrações Externas

8.1 OpenRouter (LLM Principal)

Base URL: `https://openrouter.ai/api/v1`

Auth: `Authorization: Bearer <OPENROUTER_API_KEY>`

Uso	Modelo Padrão	Variável de Configuração
Processamento principal	<code>openai/gpt-4o-mini</code>	<code>main_model</code> (system_settings)
Intent routing	<code>google/gemini-2.0-flash-lite-001</code>	<code>bridge_model</code> (system_settings)
Geração de cache (CRON)	Configurável	<code>cron_model</code> (system_settings)
Geração de prompts (admin)	<code>openai/gpt-4o</code>	Fixo no serviço

Endpoint usado: `POST /api/v1/chat/completions`

Formato de requisição:

```
{
  "model": "openai/gpt-4o-mini",
  "messages": [
    { "role": "system", "content": "<prompt do sistema>" },
    { "role": "user", "content": "<mensagem do usuário>" }
  ],
  "max_tokens": 1000
}
```

8.2 Magisterium AI (Teologia)

Base URL: `https://www.magisterium.com/api/v1`

Auth: `Authorization: Bearer <MAGISTERIUM_API_KEY>`

Modelo: `magisterium-expert`

Características:

- Especializado em Magistério da Igreja Católica
- Retorna citações com fontes (documentos, encíclicas, Catecismo)
- Usado para intenções: `THEOLOGY` , `BIBLE` , `SAINT` , `PRAYER`

Formato de resposta com citações:

```
{
  "choices": [{
    "message": {
      "content": "A doutrina da Igreja sobre...",
      "citations": [
        { "source": "Catecismo da Igreja Católica", "paragraph": 1234 }
      ]
    }
  ]
}
```

8.3 Asaas (Pagamentos)

Base URL (produção): `https://api.asaas.com/api/v3`

Base URL (sandbox): `https://sandbox.asaas.com/api/v3`

Auth: `access_token: <ASAAS_API_KEY>`

Operações realizadas:

Operação	Endpoint Asaas	Descrição
Criar cliente	<code>POST /customers</code>	Cadastrar cliente na Asaas
Criar checkout	<code>POST /checkouts</code>	Gerar link de pagamento
Buscar cliente	<code>GET /customers?cpfCnpj=...</code>	Verificar cliente existente
Criar assinatura	<code>POST /subscriptions</code>	Assinatura recorrente
Cancelar assinatura	<code>DELETE /subscriptions/:id</code> ou <code>PATCH</code>	Cancelar cobrança recorrente
Listar cobranças	<code>GET /payments?subscription=...</code>	Histórico de pagamentos

8.4 UAZAPI (WhatsApp)

Base URL: `<UAZAPI_INSTANCE_URL>`

Auth: `token: <UAZAPI_INSTANCE_TOKEN>`

Operações realizadas:

Operação	Endpoint	Body	Descrição
Enviar texto	<code>POST /send/text</code>	<code>{ phone, text }</code>	Mensagem simples
Enviar botões	<code>POST /send/interactive</code>	<code>{ phone, buttons[], text }</code>	Botões interativos
Marcar lida	<code>POST /chat/read</code>	<code>{ chatId }</code>	Marcar conversa como lida
Indicador digitação	<code>POST /message/presence</code>	<code>{ chatId, presence: "composing" }</code>	Simular "digitando..."

Lógica de fallback: Se botões interativos falharem, reenviar como lista numerada em texto.

8.5 Nodemailer (SMTP)

Uso: Envio de e-mail transacional com código de ativação

Config: Via variáveis `SMTP_*`

Template de e-mail enviado:

- Assunto: "Seu código de ativação MarIA"
 - Corpo: HTML com código `MARIA-XXXXXX` e instruções
 - Fallback: Se SMTP não configurado, log no console (modo dev)
-

8.6 AwesomeAPI (Taxa de Câmbio)

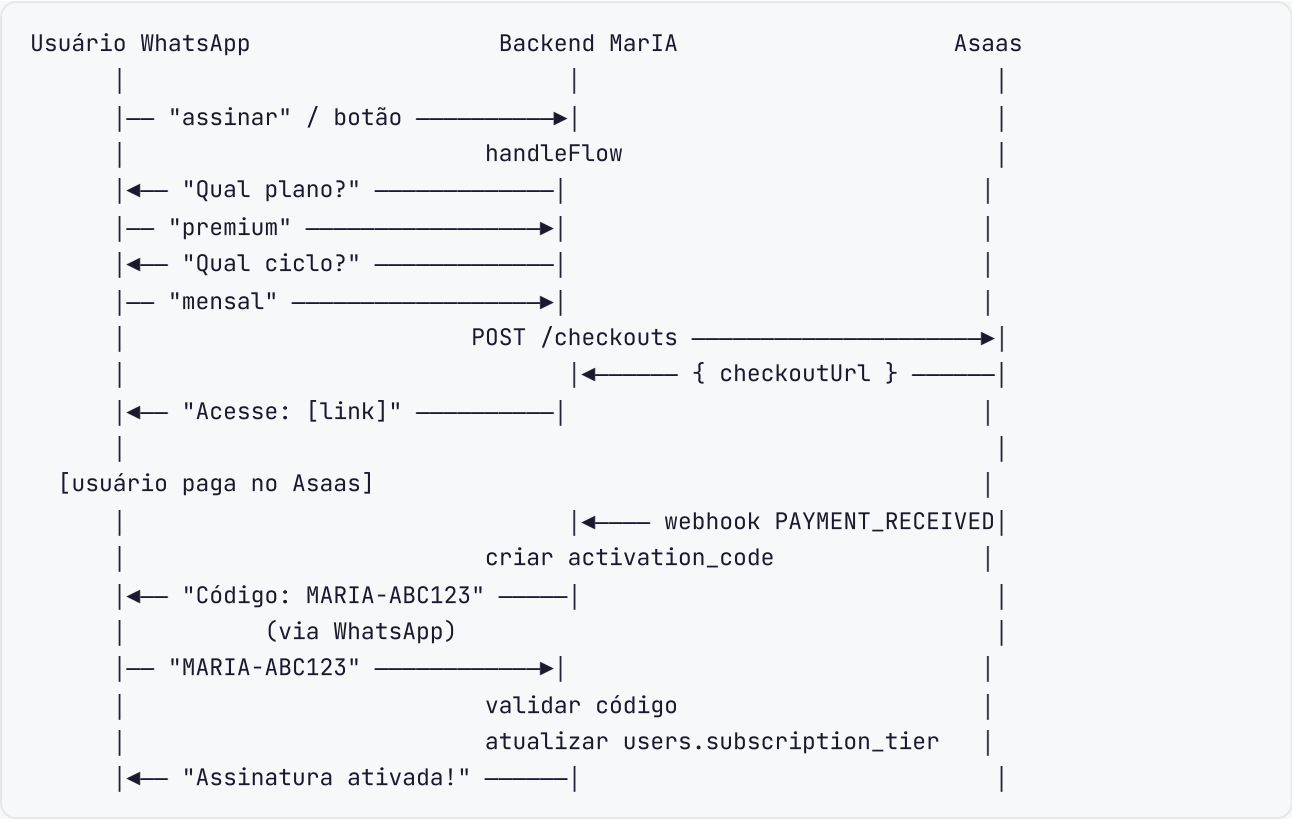
Uso: Sincronização da taxa BRL/USD para cálculo de custos

Trigger: Botão "Sync Exchange" no painel de configurações

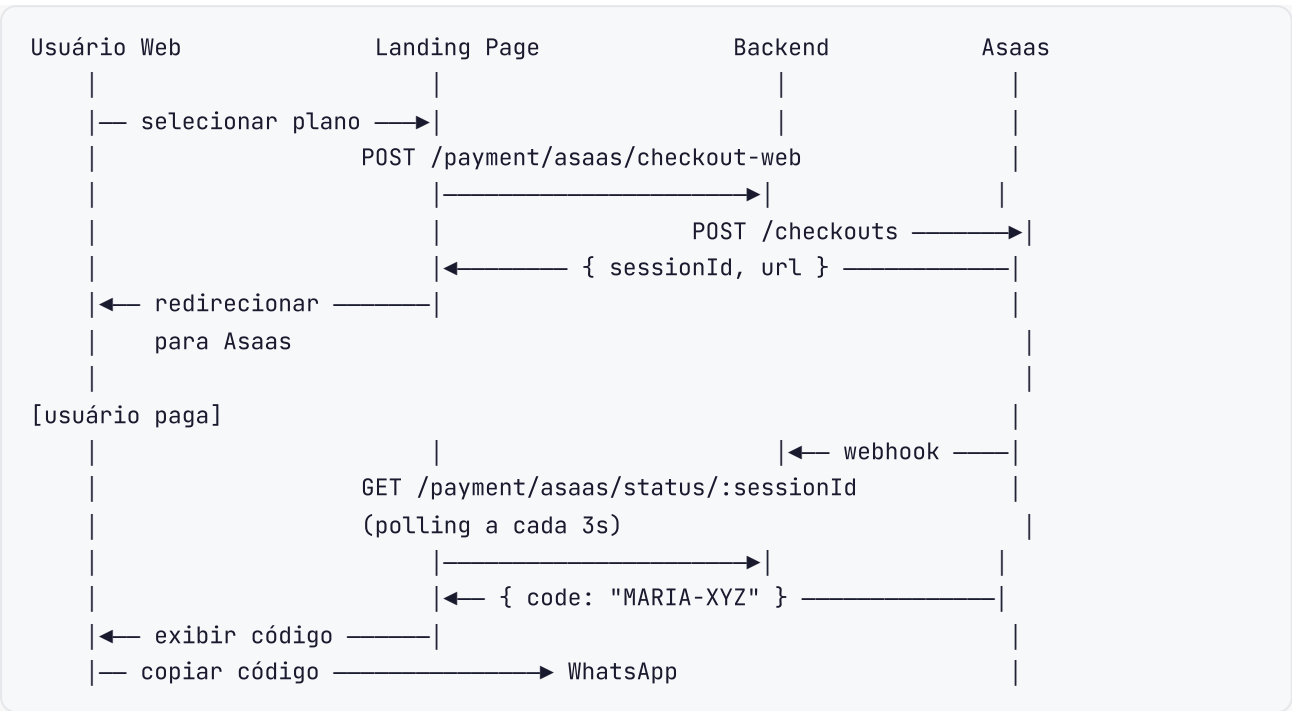
Storage: `system_settings` onde `key = 'brl_rate'`

9. Fluxos de Dados

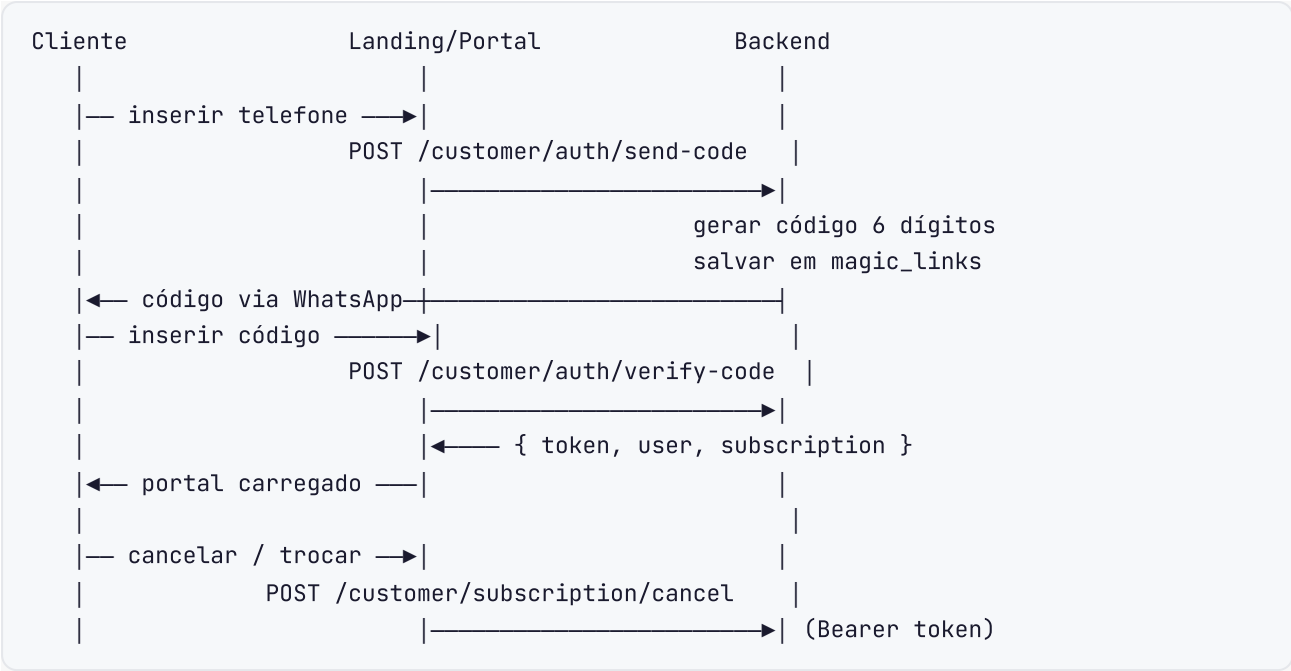
9.1 Fluxo de Assinatura via WhatsApp



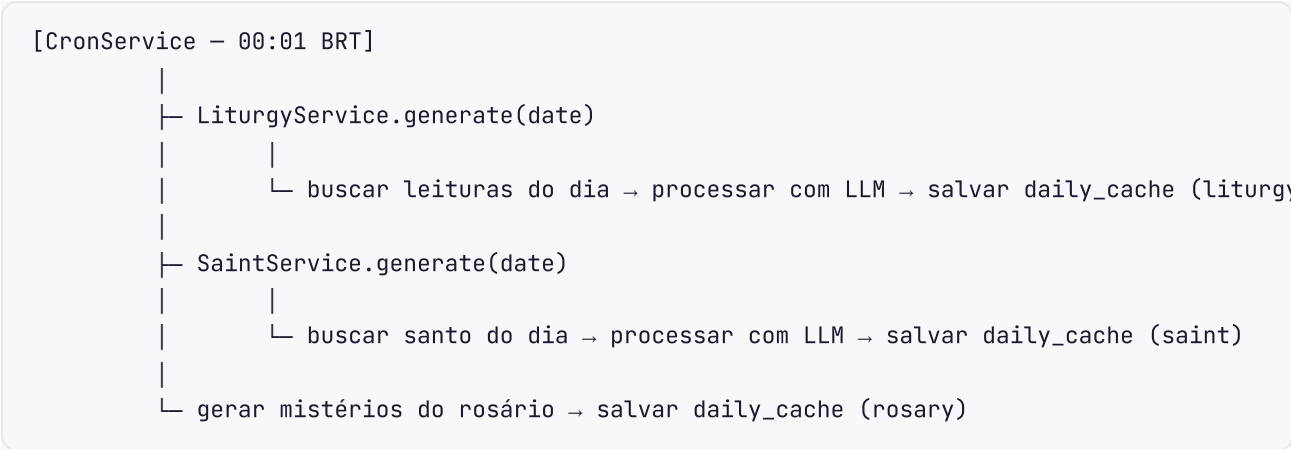
9.2 Fluxo de Assinatura via Web



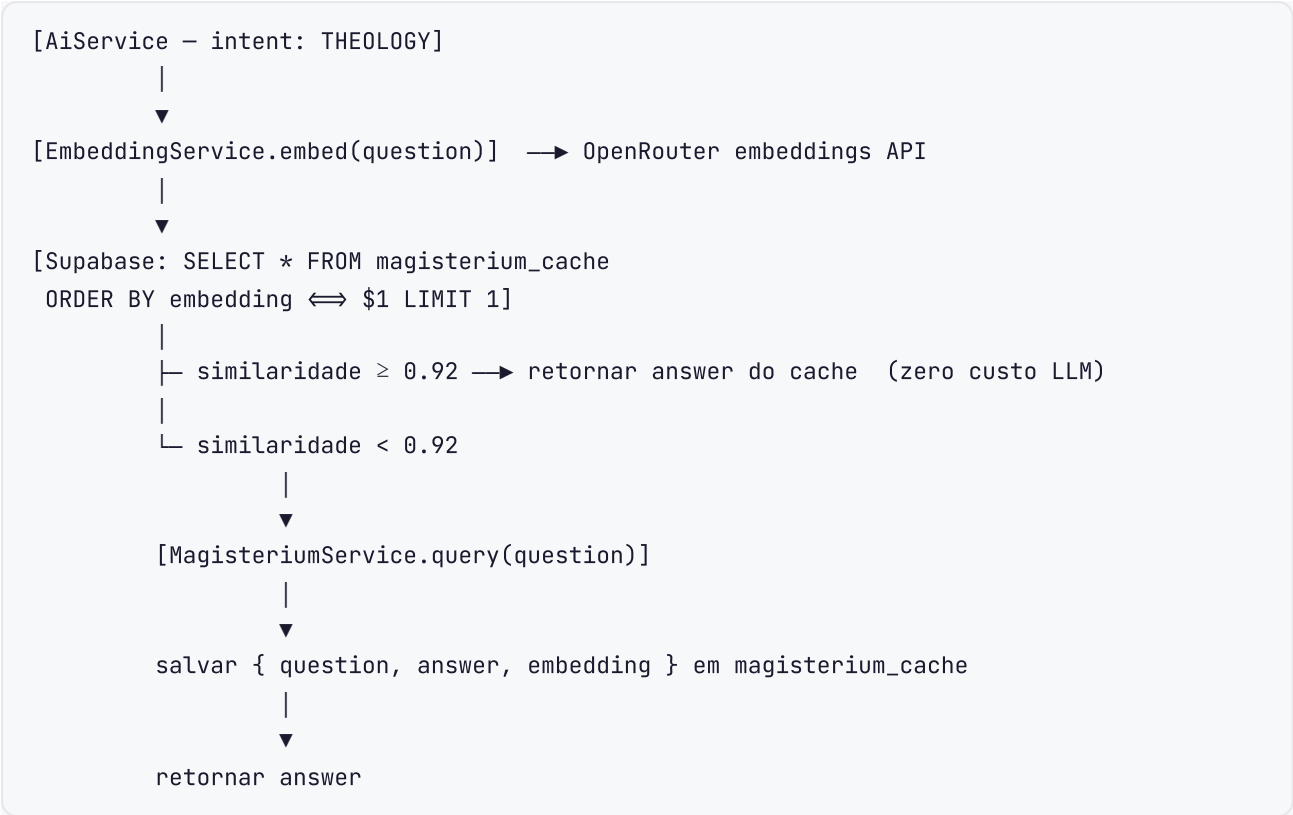
9.3 Fluxo de Autenticação no Portal



9.4 Geração de Cache Diário (CRON)



9.5 Fluxo de Cache Semântico



10. Autenticação e Autorização

10.1 Admin

Mecanismo	Detalhe
Provedor	Supabase Auth (email + senha)
Header de autorização	<code>x-admin-id: <uuid></code>
Primeiro login	Redireciona para troca obrigatória de senha
Roles	<code>superadmin</code> (acesso total), <code>admin</code> (acesso padrão)
Auditoria	Toda ação registrada em <code>activity_logs</code>

Operações restritas a `superadmin` :

- Excluir admins
- Limpar cache semântico
- Ações destrutivas em usuários

10.2 Cliente (Portal)

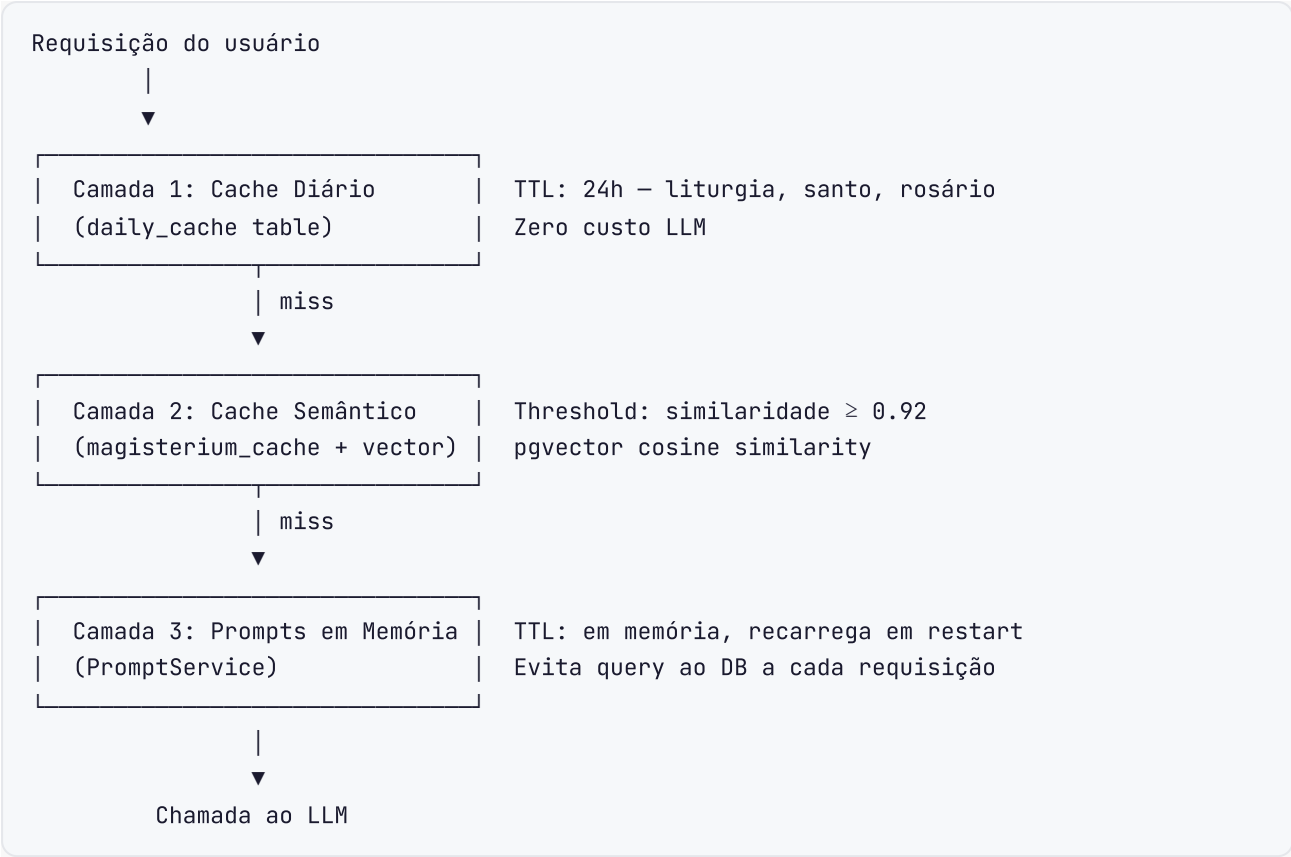
Mecanismo	Detalhe
Método atual	Código de 6 dígitos enviado via WhatsApp
Método legado	Magic link via WhatsApp
Storage do token	Tabela <code>magic_links</code> com TTL
Header de autorização	<code>Authorization: Bearer <token></code>
Rate limit	5 req/min verificação · 3 req/min envio

10.3 Segurança de API

Endpoint	Rate Limit	Auth
<code>/auth/login</code>	3 req/min	Pública
<code>/customer/auth/send-*</code>	3 req/min	Pública
<code>/customer/auth/verify*</code>	5 req/min	Pública
<code>/admin/*</code>	10 req/min	x-admin-id
<code>/ai/*</code>	10 req/min	Nenhuma (interna)
<code>/payment/asaas/webhook</code>	Sem limite	Token no header
<code>/webhook/uazapi</code>	Sem limite	Nenhuma (IP restrito)

11. Sistema de Cache

11.1 Três Camadas



11.2 Prompt Keys

Key	Descrição
core_persona	Personalidade e tom da MarIA
intent_router	Classificação de intenções
extractor_name	Extração de nome do usuário
context_condenser	Resumo de contexto
liturgy_generator	Geração de conteúdo litúrgico
saint_generator	Geração do santo do dia

12. Sistema de Cotas e Assinaturas

12.1 Tiers e Limites

Tier	Mensagens LLM/mês	Acesso	Observação
free	0	Cache apenas	Novo usuário ou expirado
basic	300	LLM + Cache	Plano pago
premium	600	LLM + Cache	Plano pago
unlimited	∞	Tudo	Promoções / manual
admin	∞	Tudo	Equipe interna

12.2 Verificação de Quota

```
// Fluxo em AiService
1. contar messages WHERE user_id = X AND is_llm = true AND created_at > start_of_month
2. comparar com limite do tier (300 ou 600)
3. se ultrapassou [] responder com mensagem de limite
4. se monthly_limit_brl definido [] calcular custo e comparar
```

12.3 Reset Mensal

O reset ocorre automaticamente pela data de expiração: `subscription_expires_at` é avançado mensalmente pelo webhook Asaas a cada renovação bem-sucedida.

13. Variáveis de Ambiente

13.1 Backend (**backend/.env**)

```
# Servidor
PORT=3000

# Supabase
SUPABASE_URL=https://xxx.supabase.co
SUPABASE_KEY=eyJ... # Chave anon (client) OU service role
SUPABASE_SERVICE_ROLE_KEY=eyJ... # Chave para operações privilegiadas

# OpenRouter (LLM)
OPENROUTER_API_KEY=sk-or-...
MAIN_MODEL=openai/gpt-4o-mini # Sobrescrito por system_settings.main_model
BRIDGE_MODEL=google/gemini-2.0-flash-lite-001

# Magisterium AI
MAGISTERIUM_API_KEY=...
MAGISTERIUM_API_URL=https://www.magisterium.com/api

# Asaas (Pagamentos)
ASAAS_API_KEY=...
ASAAS_API_URL=https://sandbox.asaas.com/api/v3 # trocar para prod
ASAAS_AUTH_TOKEN=... # Token validado no webhook
ASAAS_CHECKOUT_SUCCESS_URL=https://maria.acutistech.com.br/?checkout=success
ASAAS_CHECKOUT_CANCEL_URL=https://maria.acutistech.com.br/?checkout=cancel

# UAZAPI (WhatsApp)
UAZAPI_INSTANCE_URL=https://...
UAZAPI_INSTANCE_TOKEN=...

# SMTP (E-mail)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=...
SMTP_PASS=...
SMTP_FROM="MarIA" <noreply@maria.acutistech.com.br>

# URLs do sistema
FRONTEND_URL=https://maria.acutistech.com.br
```

13.2 Admin Frontend (**frontend/.env.production**)

```
VITE_API_URL=https://api.maria.acutistech.com.br
```

13.3 Landing Page (`landing/.env.production`)

NEXT_PUBLIC_API_URL=https://api.maria.acutistech.com.br

14. Frontend — Painel Administrativo

14.1 Páginas e Rotas

Rota	Página	Descrição
<code>/</code>	<code>login.tsx</code>	Login do admin
<code>/dashboard</code>	<code>dashboard.tsx</code>	Overview com estatísticas e conversas recentes
<code>/users</code>	<code>users.tsx</code>	Listagem de admins (legacy)
<code>/wa-users</code>	<code>wa-users.tsx</code>	Usuários WhatsApp com filtros e ações
<code>/ai-settings</code>	<code>ai-settings.tsx</code>	Prompts e configuração de modelos
<code>/flows</code>	<code>flows.tsx</code>	Fluxos automáticos (builder visual)
<code>/prayers</code>	<code>prayers.tsx</code>	Conteúdo de orações
<code>/daily-content</code>	<code>daily-content.tsx</code>	Cache de liturgia, santo, rosário
<code>/finance</code>	<code>finance.tsx</code>	Dashboard financeiro e assinaturas
<code>/settings</code>	<code>settings.tsx</code>	Configurações globais do sistema
<code>/logs</code>	<code>logs.tsx</code>	Logs de uso e webhooks

14.2 Componentes Principais

```
components/
├─ layout/
│   ├─ main-layout.tsx      # Wrapper com sidebar + header
│   └─ sidebar.tsx          # Navegação lateral com ícones
│
├─ dashboard/
│   └─ stats-cards.tsx      # Cards de KPI (usuários, receita, tokens)
│
└─ ui/                      # Shadcn UI
    ├─ button, card, badge, avatar
    ├─ input, label, textarea
    ├─ dialog, dropdown-menu, tabs
    ├─ table, sonner (toast)
    └─ ...
```

14.3 Comunicação com API

Todas as chamadas passam pelo wrapper em `lib/api.ts` que:

- Injeta `VITE_API_URL` como base
- Injeta automaticamente `x-admin-id` do `localStorage`
- Trata erros HTTP globalmente

15. Landing Page

15.1 Estrutura (Next.js App Router)

```
landing/src/app/
├── page.tsx           # Página principal: hero, planos, FAQ
├── portal/            # Portal do cliente
│   └── page.tsx       # Login + gestão de assinatura
└── layout.tsx         # Layout global
```

15.2 Fluxo de Checkout Web

1. Usuário seleciona plano na landing page
2. `POST /payment/asaas/checkout-web` → recebe `{ sessionId, checkoutUrl }`
3. Redirecionar para `checkoutUrl` (Asaas)
4. Polling `GET /payment/asaas/status/:sessionId` a cada 3s
5. Quando `status = 'paid'`, exibir código `MARIA-XXXXXX`
6. Botão "Abrir WhatsApp" com código pré-preenchido na URL

15.3 Portal do Cliente

1. Inserir número de telefone
2. `POST /customer/auth/send-code` → código chega via WhatsApp
3. Inserir código → `POST /customer/auth/verify-code` → `{ token }`
4. `GET /customer/subscription/status` (Bearer token) → dados da assinatura
5. Ações disponíveis: cancelar, trocar plano

16. Segurança

16.1 Medidas Implementadas

Área	Medida
Banco de dados	Row Level Security (RLS) no Supabase
Admin	Senha obrigatória no primeiro login

Área	Medida
Admin	Auditoria completa de ações (activity_logs)
Webhooks Asaas	Validação de token no header <code>asaas-access-token</code>
Rate limiting	Global 10 req/min; login 3 req/min; portal 5 req/min
CORS	Configurado globalmente no NestJS
Dados sensíveis	API keys apenas em <code>.env</code> (nunca commitadas)
Deleção de usuário	Suporte a purge completo (mensagens, contexto, assinatura)

16.2 Dados Protegidos por RLS

- `users` — acesso restrito ao próprio registro
- `messages` — acesso restrito por `user_id`
- `subscriptions` — acesso restrito por `user_id`
- `magic_links` — acesso restrito por `user_id`

O backend usa `service_role_key` para operações privilegiadas que precisam contornar o RLS.

16.3 Verificação de Webhook Asaas

```
// asaas.controller.ts
const token = request.headers['asaas-access-token'];
if (token !== process.env.ASAAS_AUTH_TOKEN) {
  throw new UnauthorizedException('Invalid webhook token');
}
```

17. Observabilidade e Logs

17.1 Tabelas de Log

Tabela	Conteúdo	Retenção
<code>usage_logs</code>	Tokens, modelo, custo por mensagem	Indefinida
<code>webhook_logs</code>	Payload de webhooks recebidos	Indefinida
<code>activity_logs</code>	Ações administrativas (quem fez o quê, quando)	Indefinida

17.2 Métricas Calculadas

No painel `/finance` :

- Receita total (soma de `subscriptions.amount` WHERE `status = 'paid'`)
- Custo total (soma de `usage_logs.cost` * `system_settings.brl_rate`)

- Margem bruta e percentual
- Filtragem por período

No painel **/dashboard** :

- Total de usuários por tier
- Mensagens nas últimas 24h
- Tokens usados no período
- Conversas recentes

17.3 Jobs Automáticos (CRON)

Job	Horário	Ação
Cache diário	00:01 BRT	Gerar liturgia, santo, rosário
(futuro) Exchange rate	—	Sincronizar BRL/USD

18. Infraestrutura e Deploy

18.1 Processo em Produção (PM2)

```
// ecosystem.config.js
module.exports = {
  apps: [
    {
      name: 'maria-backend',
      script: 'dist/main.js',
      cwd: './backend',
      env: { NODE_ENV: 'production' }
    },
    {
      name: 'maria-frontend',
      script: 'serve',
      args: '-s dist -l 4173',
      cwd: './frontend'
    },
    {
      name: 'maria-landing',
      script: 'node_modules/.bin/next',
      args: 'start',
      cwd: './landing'
    }
  ]
}
```

18.2 Scripts de Desenvolvimento

```
# Windows (PowerShell)
./dev.ps1

# Ou via npm (raiz do projeto)
npm run dev          # inicia backend + frontend em paralelo
npm run install:all  # instala dependências em todos os projetos
npm run build:all    # build de produção de todos os projetos
```

18.3 Deploy

```
./deploy.sh
# 1. git pull
# 2. npm run install:all
# 3. npm run build:all
# 4. pm2 restart all
```

18.4 Portas Padrão

Serviço	Porta
Backend API	3000
Admin Frontend (dev)	5173
Admin Frontend (prod)	4173
Landing Page (dev)	3001
Landing Page (prod)	3000 (ou Nginx proxy)

18.5 Checklist de Deploy

- ☐ Variáveis de ambiente configuradas (`.env` no backend)
- ☐ `ASAAS_API_URL` apontando para produção (não sandbox)
- ☐ `SUPABASE_SERVICE_ROLE_KEY` configurada
- ☐ Webhook UAZAPI configurado para apontar para `https://api.dominio.com/webhook/uazapi`
- ☐ Webhook Asaas configurado para apontar para `https://api.dominio.com/payment/asaas/webhook`
- ☐ `ASAAS_AUTH_TOKEN` igual ao configurado no painel Asaas
- ☐ CRON job ativo (CronService interno ao NestJS)
- ☐ PM2 iniciado e salvo (`pm2 save`)